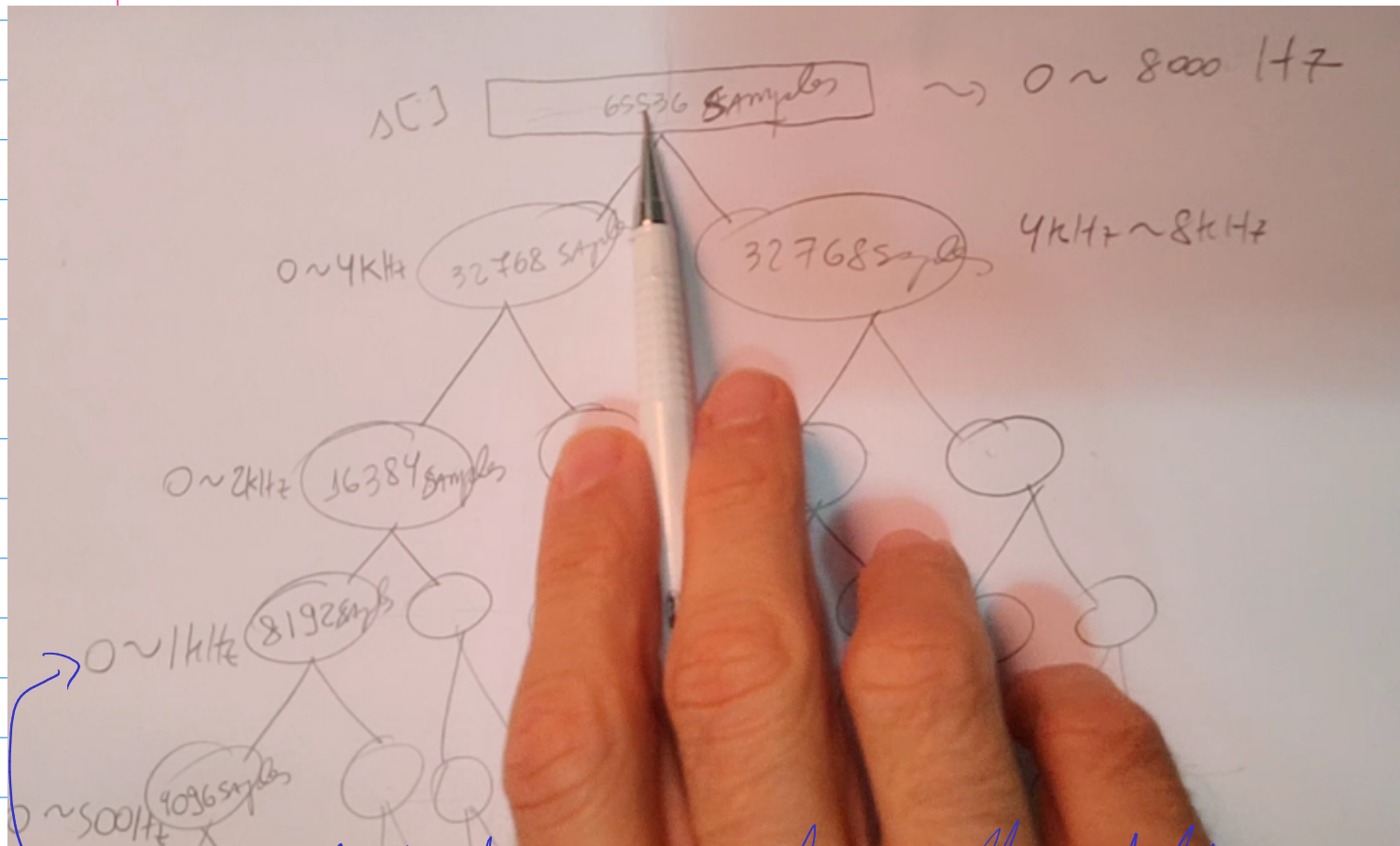


(first half)
 if we set a gain half of a high
 amplitude by gain and then we do
 the inverse DT WPT, the inverse ST-1 signal
 will have that range of frequencies is
 more.



we could try to fix the shift but
 with the signals are too long it
 wouldn't affect us

The filter we would choose for
 this DT WPT application follows the
 same idea as before. The more we want
 to decrease the length the filter
 should be.

We could also get part of the signal, process it with a tapered filter, & the instruction apply it on the value (constant) and the part it back. It's not a weight process, and we need to meet the filter, but at the same time we don't need to use FFT's algorithm.

Then you change the input, when you get the input you have [512] looking more closely to what the reading weight function looks like.

As long as you can solve the same rule.
We could use different thresholds.

Hard: if the amplitude is non zero $\leq T$, set it to 0

Soft: if $\leq T$ set it to 0 otherwise if $> T$ set it to $\text{val} - T$ and if $< -T$ set it to $\text{val} + T$

T can be equivalently found but also

on the universal threshold:

hard thresholding. Hard threshold, however, provides better edge preservation in comparison with the soft one.

- Donoho and Johnstone developed an universal threshold for applications involving wavelets. It provides an easy and simple way to find the best threshold T for noise removal. Once a decomposition tree is ready, the threshold for each leaf is $T = \frac{\text{MAD}}{0.6745} \sqrt{2 \log(b)}$ where $\text{MAD} = \text{median}(|\text{DTWPT}_j(s[\cdot])[i] - \text{median}(\text{DTWPT}_j(s[\cdot])[i])|)$ is the median absolute deviation in the leaf with b elements.

163/200

--- Application in REVISIONED: lossy compression ---

- **Numerical Example:** if a hypothetical DTWPT has in one particular leaf the values $\{1, 2, 9, 1, 4, 6, 2, 10\}$, then, to find the median, we first sort the data from the least to the greatest, i.e., $\{1, 1, 2, 2, 4, 6, 9, 10\}$. Proceeding, we simply choose the central element as being the median. Nevertheless, since wavelet tree leaves always have an even number of elements as in this example, we compute the median by taking the mean of the two middlemost numbers, i.e., $\frac{2+4}{2} = 3$. Now, we subtract 3 from each original element in the leaf, producing $\{1-3, 2-3, 9-3, 1-3, 4-3, 6-3, 2-3, 10-3\} = \{-2, -1, 6, -2, 1, 3, -1, 7\}$. The corresponding absolute values are $\{2, 1, 6, 2, 1, 3, 1, 7\}$. Lastly, we obtain the median once again by sorting the data, producing $\{1, 1, 1, 2, 2, 3, 6, 7\}$, and calculating the mean of the two middlemost numbers, i.e., $\frac{2+2}{2} = 2$. Thus, $\text{MAD} = 2$.
- MAD is a measure of statistical dispersion, being more resilient to outliers in a data set than the standard deviation.

164/200

the
idea
is that
the values are normally distributed

the values are normally distributed

Hard can introduce artifacts
Soft is smoothing

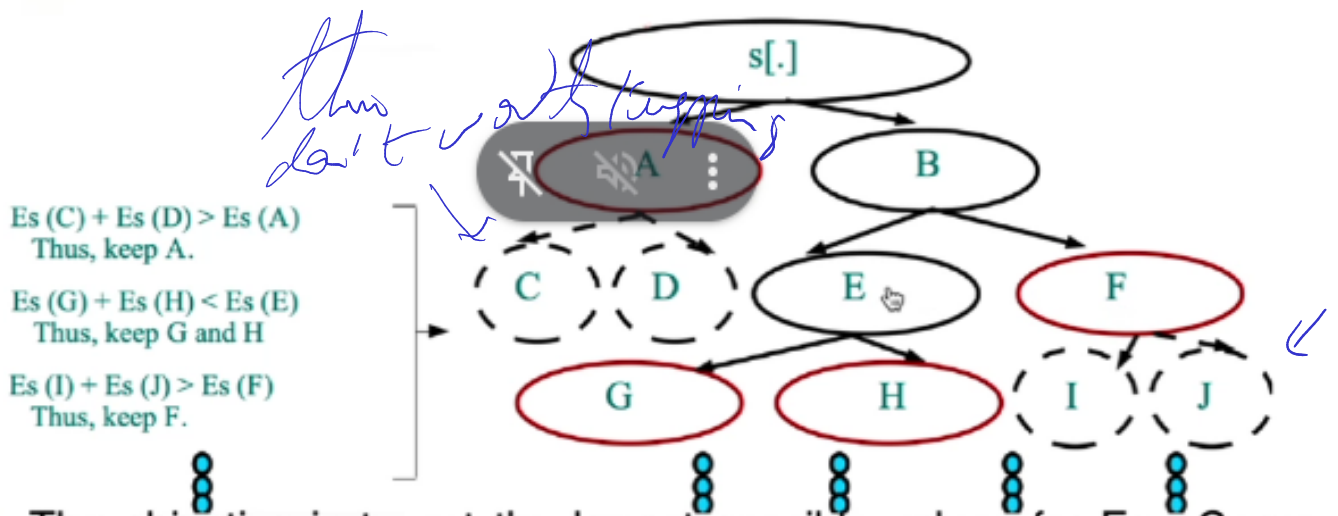
- - - Application III REVISITED: lossy compression - - -

- ▶ Another interesting possibility for lossy compression is the adoption of a strategy known as *best basis*. It consists of finding the best decomposition tree, used to represent a certain input signal $s[\cdot]$, which is usually obtained based on the criterion of **entropy** (E_s). The best tree might be different of ordinary DTWT or $\overline{\text{DTWPT}}$ trees, implying on irregular decompositions. Based on a certain filter bank, the basic rule is as follows:
 - ▶ $\text{DTWPT}_1(s[\cdot])$ is calculated;
 - ▶ if $(\text{entropy}(\text{left_leaf}) + \text{entropy}(\text{right_leaf})) < \text{entropy}(s[\cdot])$, then keep the leaves and go to the next level, otherwise, cancel the transformation and go back to the original signal;
- ▶ Then, the process is recursively repeated, for all the subsequent leaves in comparison to their fathers: if $(\text{entropy}(\text{left_child}) + \text{entropy}(\text{right_child})) < \text{entropy}(\text{father_node})$, then keep the children and go to the next level, otherwise, cancel the transformation for that father node.

Initially you would be in the middle level to compare. Then, you would use the entropy that would give you for the branches and you should follow the tree can be best complete in other cases.

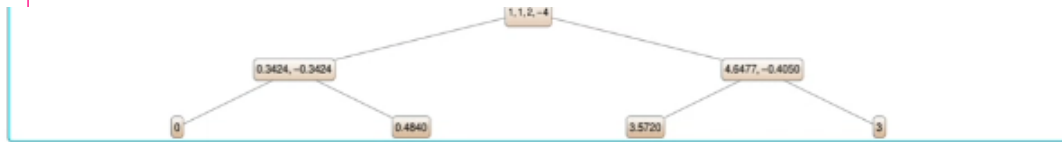
- - - Application III REVISITED: lossy compression - - -

- ▶ The following example shows exactly the strategy adopted to find the best basis decomposition:



- ▶ The objective is to get the lowest possible values for Es. Consequently, the final tree can be irregular...
- ▶ A low value for Es can be understood, from a practical perspective, as a low amount of data required to represent information.

In the wild node values are not quite equal to each other. So that, there's an alternative way to merge, or just to take the quantity the number of values. This alternative method is based on entropy, but instead of giving probability we can use the weighted sum (entropy is not the best). That's the method usually used to find the best tree, but we're trying to find the best thing.



For the **left** part of the tree in the second level, we have:

- ▶ $Es(\text{left node in the first level}) = 0.3010$ (from the prior slide)
- ▶ $Es(\text{left leaf in the second level}) = 0$
- ▶ $Es(\text{right leaf in the second level}) = \frac{(0.4840)^2}{(0.4840)^2} \log\left(\frac{(0.4840)^2}{(0.4840)^2}\right) = 1 \cdot 0 = 0$

Thus, $0 + 0 = 0 < 0.3010$. Therefore, we keep the children nodes.

For the **right** part of the tree in the second level, we have:

- ▶ $Es(\text{left node in the first level}) = 0.0480$ (from the prior slide)
- ▶ $Es(\text{left leaf in the second level}) = \frac{(3.5720)^2}{(3.5720)^2} \log\left(\frac{(3.5720)^2}{(3.5720)^2}\right) = 1 \cdot 0 = 0$
- ▶ $Es(\text{right leaf in the second level}) = \frac{(3)^2}{(3)^2} \log\left(\frac{(3)^2}{(3)^2}\right) = 1 \cdot 0 = 0$

Thus, $0 + 0 = 0 < 0.0480$. Therefore, we keep the children nodes.



Wavelet Transform and its Applications

when the entropy of the children is $<$ then that parent can not continue

Ex feature extraction

- ↳ apply DWT on the signal and find the coefficients
- ↳ is just necessary to know the position of the frequencies (wave), that's why we should know the decomp. level and then filter
- ↳ we could extract → energy, entropy, go higher (they can indicate a maximum or min. part), variance, etc.

↳ Usually a large input
needs to be transformed to get
features in a fixed input
the dimension.
↳ For DNN feature extraction
is usually embedded

↳ autoencoder can be used to
find features by training input =
output and then writing the
net or help to get the features.
However the physical characteristic
can be lost.

If we want to remove frequencies from the whole signal SE , we need to use all values as a help to give. Otherwise, if we want to remove from an specific part of SE we can simply set some of the values to 0.

- Best basis → but disadvantage due to
- ↳ represent a signal SE
 - ↳ Entropy
 - ↳ Entropy left and right must be
↳ ^{father} ~~father~~
 - ↳ get the largest Entropy (has argument of data to represent the information).